



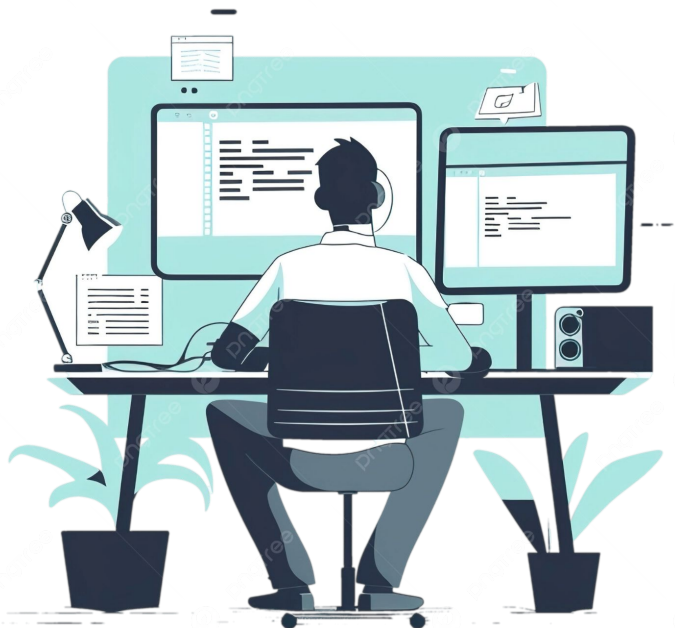
MODULO VERTICALE

UI Programming

Lezione 1



COSA SIGNIFICA E COSA COMPRENDE UI PROGRAMMING



UI Programmer

Per UI Programming si intende tutta la parte di programmazione relativa alla creazione dei sistemi di interfaccia.

Questi sistemi verranno successivamente utilizzati dai UI Designer e UI Artist per creare l'effettivo collegamento tra il gameplay e l'utente.

Lo UI Programmer non esiste solo nell'ambito della produzione di videogiochi, ma esiste anche in altri ambienti dove spesso però è accorpato sotto il nome di UI Developer



Esempi di Sistemi

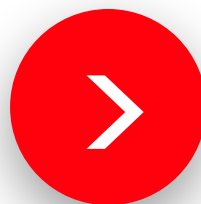
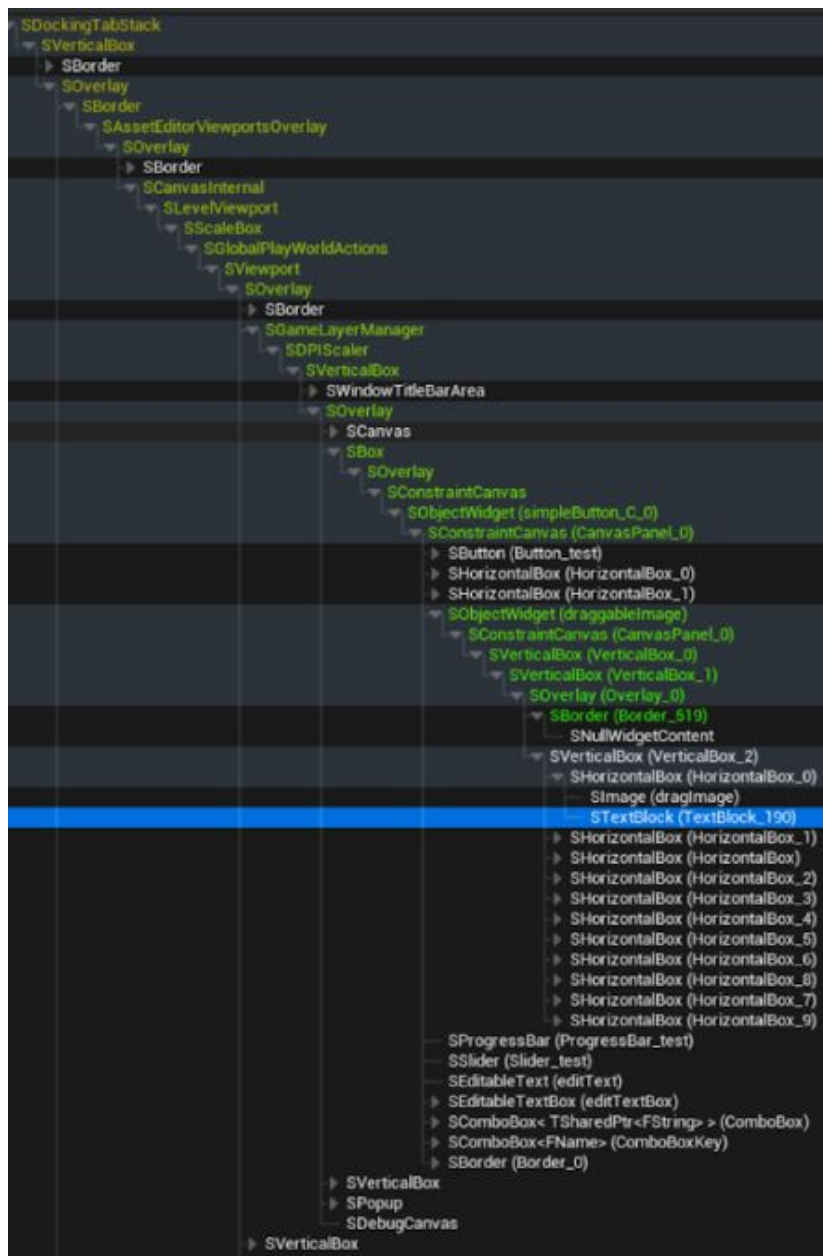
- Menu
- HUD
- Inventari
- Schermate di raccordo (Loading etc...)
- Interfacce di dialogo
- Mappe
- Control bar/Callout per gli input
- Sistemi di mapping tra dati e grafica
- UI Manager





UNREAL UI

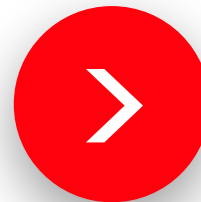
SLATE



Cos'è Slate

Slate è un framework di interfaccia utente completamente personalizzato e indipendente dalla piattaforma (Windows, Console, Linux ecc), progettato per rendere divertente (mica tanto) ed efficiente la creazione di interfacce utente.

Combina una sintassi dichiarativa con la capacità di progettare, disporre e definire facilmente componenti che consentono di creare e iterare facilmente sulle interfacce utente

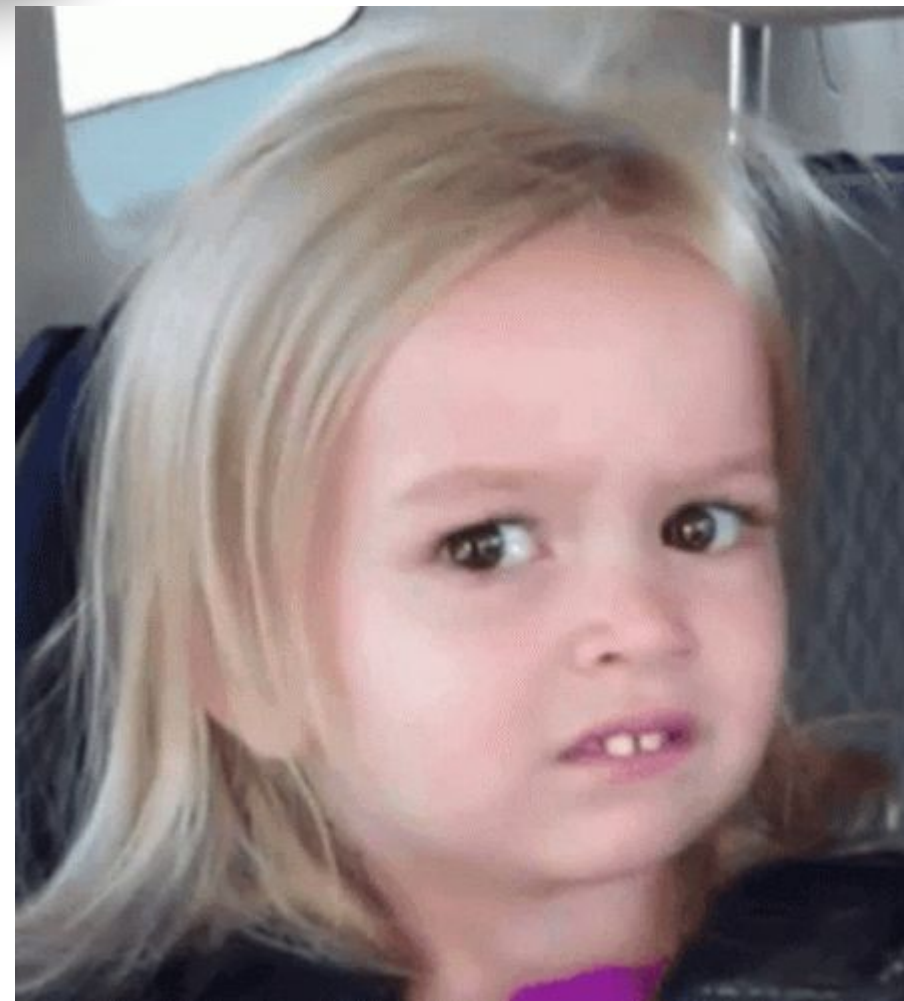


Slate è complesso?

Il framework di Slate può sembrare complesso e non sempre vale la pena averci a che fare, ma dipendentemente a cosa bisogna sviluppare, come tutto, ha i suoi pro e contro.

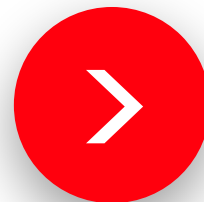
Slate è il compagno d'avventura perfetto (e probabilmente l'unico di cui fidarsi) quando si ha a che fare con l'editor stesso di Unreal, ma anche quando esiste la necessità di creare widget più complessi e ottimizzati per il gameplay.

Ma come è funziona Slate?





Sintassi Dichiarativa

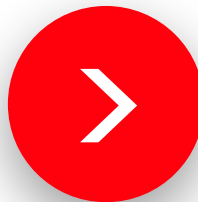


La sintassi dichiarativa di Slate rende la creazione di UI accessibile ai programmatori senza aggiungere livelli di indirectione.

Viene fornito un set completo di macro per ""semplificare"" il processo di dichiarazione e creazione di nuovi widget.

Spesso ridondante nella lettura.
La nomenclatura di variabili e funzione in Slate è molto importante per non perdersi

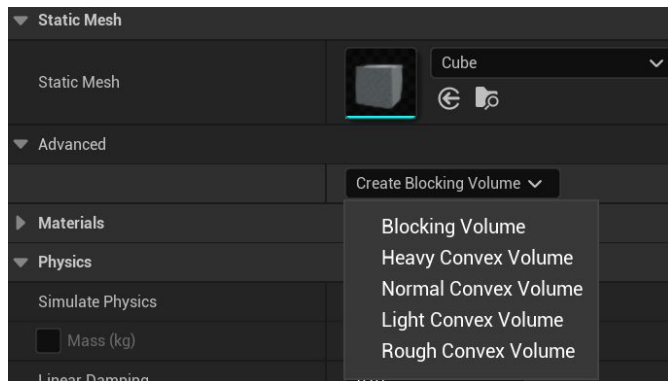
```
SLATE_BEGIN_ARGS( SSubMenuButton )
: _ShouldAppearHovered( false )
{}
/** The label to display on the button */
SLATE_ATTRIBUTE( FString, Label )
/** Called when the button is clicked */
SLATE_EVENT( FOnClicked, OnClicked )
/** Content to put in the button */
SLATE_NAMED_SLOT( FArguments, FSimpleSlot, Content )
/** Whether or not the button should appear in the hovered state */
SLATE_ATTRIBUTE( bool, ShouldAppearHovered )
SLATE_END_ARGS()
```

Composizione

Slate è basato sulla composizione dei singoli componenti (widget).

La composizione semplifica la riorganizzazione rapida degli elementi dell'interfaccia utente per la prototipazione e l'iterazione.



```
// Add a new section for static meshes
ContextualEditingWidget->AddSlot()
.Padding( 2.0f )
[
    SNew( SDetailSection )
    .SectionName("StaticMeshSection")
    .SectionTitle( LOCTEXT("StaticMeshSection", "Static Mesh").ToString() )
    .Content()
    [
        SNew( SVerticalBox )
        + SVerticalBox::Slot()
        .Padding( 3.0f, 1.0f )
        [
            SNew( SHorizontalBox )
            + SHorizontalBox::Slot()
            .Padding( 2.0f )
            [
                SNew( SComboButton )
                .ButtonContent()
                [
                    SNew( STextBlock )
                    .Text( LOCTEXT("BlockingVolumeMenu", "Create Blocking Volume") )
                    .Font( FontInfo )
                ]
                .MenuContent()
                [
                    BlockingVolumeBuilder.MakeWidget()
                ]
            ]
        ]
    ]
];
```



Stili



Per “**Stile**” si intende :

- Stati visivi di un button
- Tipologia di una checkbox
- Tipo e grandezza di un font
- Ecc.

Gli stili possono essere creati e applicati alle varie parti di un widget.

Ciò semplifica l'iterazione dell'aspetto dei componenti nell'interfaccia utente, nonché la condivisione e il riutilizzo degli stili.

```
// Tool bar
{
    Set( "ToolBar.Background", FSlateBoxBrush( TEXT("Common/GroupBorder"), FMargin(4) );

    Set( "ToolBarButton.Normal", FSlateNoResource() );    // Note: Intentionally transparent
    Set( "ToolBarButton.Pressed", FSlateBoxBrush( TEXT("Old/MenuItemButton_Pressed") );
    Set( "ToolBarButton.Hovered", FSlateBoxBrush( TEXT("Old/MenuItemButton_Hovered") );

    // Tool bar buttons are sometimes toggle buttons, so they need styles for "checked" state
    Set( "ToolBarButton.Checked", FSlateBoxBrush( TEXT("Old/MenuItemButton_Pressed") );
    Set( "ToolBarButton.Checked_Hovered", FSlateBoxBrush( TEXT("Old/MenuItemButton_Hovered") );
    Set( "ToolBarButton.Checked_Pressed", FSlateBoxBrush( TEXT("Old/MenuItemButton_Pressed") );

    // Tool bar button label font
    Set( "ToolBarButton.LabelFont", FSlateFontInfo( TEXT("Roboto-Regular"), 8 ) );
}
```

```
SNew(SButton)
.Style(TEXT("ToolBarButton"))
.OnClicked( InKismet2.ToSharedRef(), &FKismet::Compile_OnClicked )
.IsEnabled( InKismet2.ToSharedRef(), &FKismet::InEditingMode )
.Content()
```



Input



Slate supporta l'accettazione di input da mouse e tastiera.

Incorpora un sistema flessibile di associazione di tasti che consente di associare qualsiasi combinazione di tasti a qualsiasi comando.



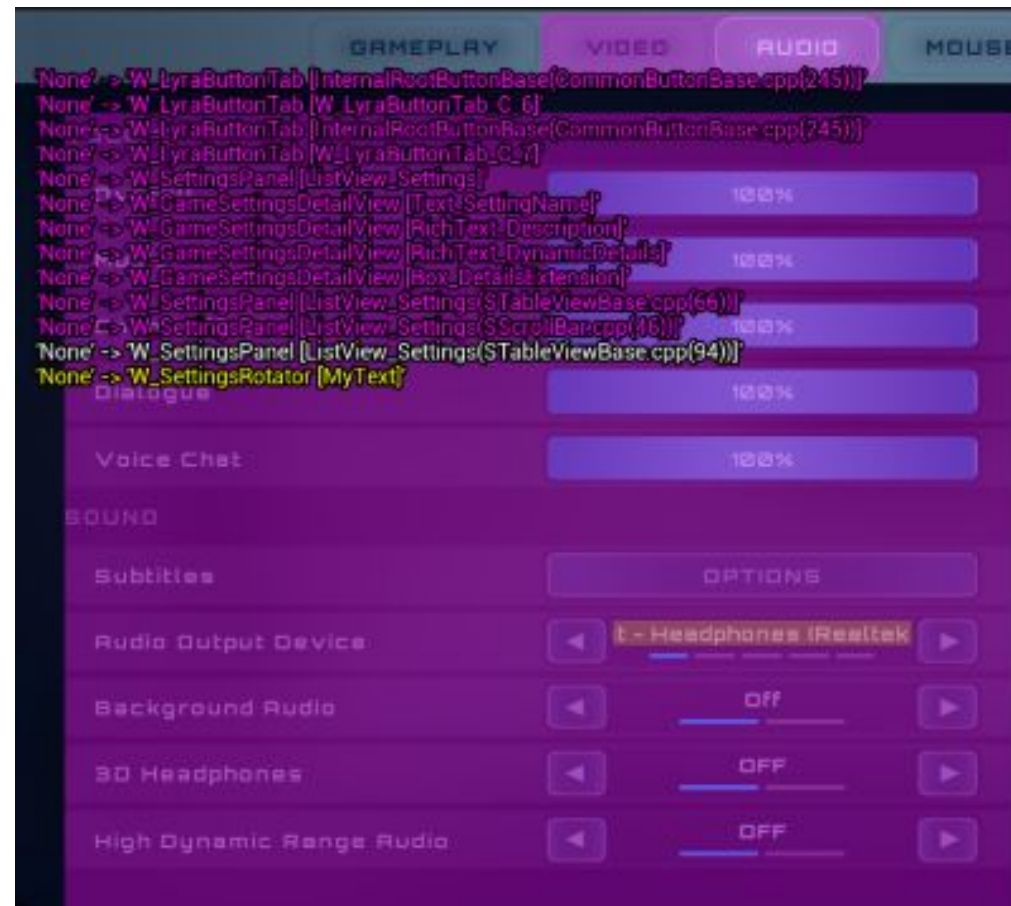


Output



Slate utilizza primitive di rendering target agnostic che gli consentono di funzionare potenzialmente su qualsiasi piattaforma.

Ha come target l'interfaccia hardware di rendering (**RHI**) di Unreal Engine, di conseguenza può essere usato su qualsiasi piattaforma su cui funziona Unreal.



Lyra Slate Debug on PS5



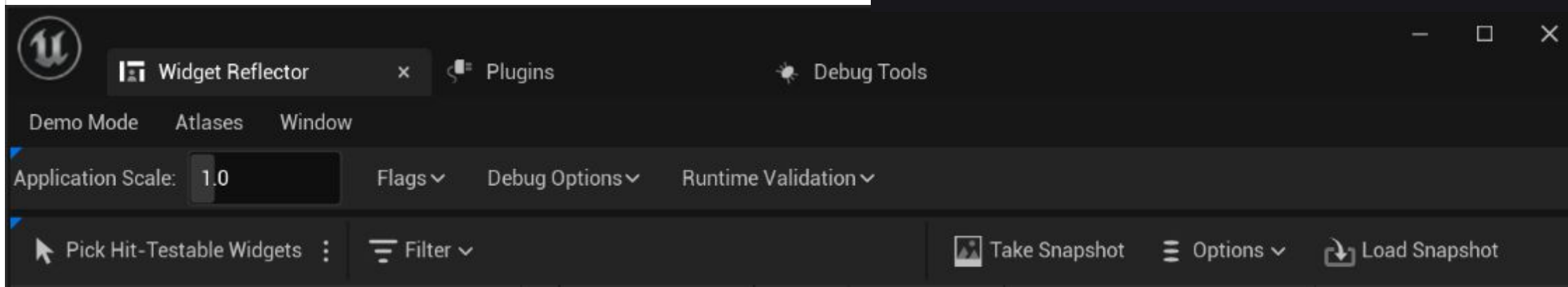
Magia nera



Slate include anche una parte di magia nera per creare layout statici.

Sotto è rappresentato il layout creato usando la semplicissima e intuitiva stringa a destra.

```
FString DefaultLayout =  
    TEXT( " {" )  
    TEXT( "   \"type\": \"tabstack\", \" )  
    TEXT( "   \"sizecoeff\": 1, \" )  
    TEXT( "   \"children\": \" )  
    TEXT( "   [" )  
    TEXT( "       {" )  
    TEXT( "           \"type\": \"tab\", \" )  
    TEXT( "           \"content\": \"Widget Inspector Tab\"" )  
    TEXT( "       }, \" )  
    TEXT( "       {" )  
    TEXT( "           \"type\": \"tab\", \" )  
    TEXT( "           \"content\": \"Plugin Manager Tab\"" )  
    TEXT( "       }, \" )  
    TEXT( "       {" )  
    TEXT( "           \"type\": \"tab\", \" )  
    TEXT( "           \"content\": \"Debug Tools\"" )  
    TEXT( "       } \" )  
    TEXT( "   ] \" )  
    TEXT( " } " );
```





Pro e Contro

Pro

- Performance derivanti dall'essere interamente in C++
- Utilizzabile per l'uso generico delle applicazioni (un esempio è che il Launcher di Epic Game e l'Unreal Editor stesso sono interamente realizzati con Slate)
- Utilizzato per estendere l'editor (ciò è dovuto al fatto che l'Unreal Editor è stato realizzato in primo luogo con Slate)
- Platform independent

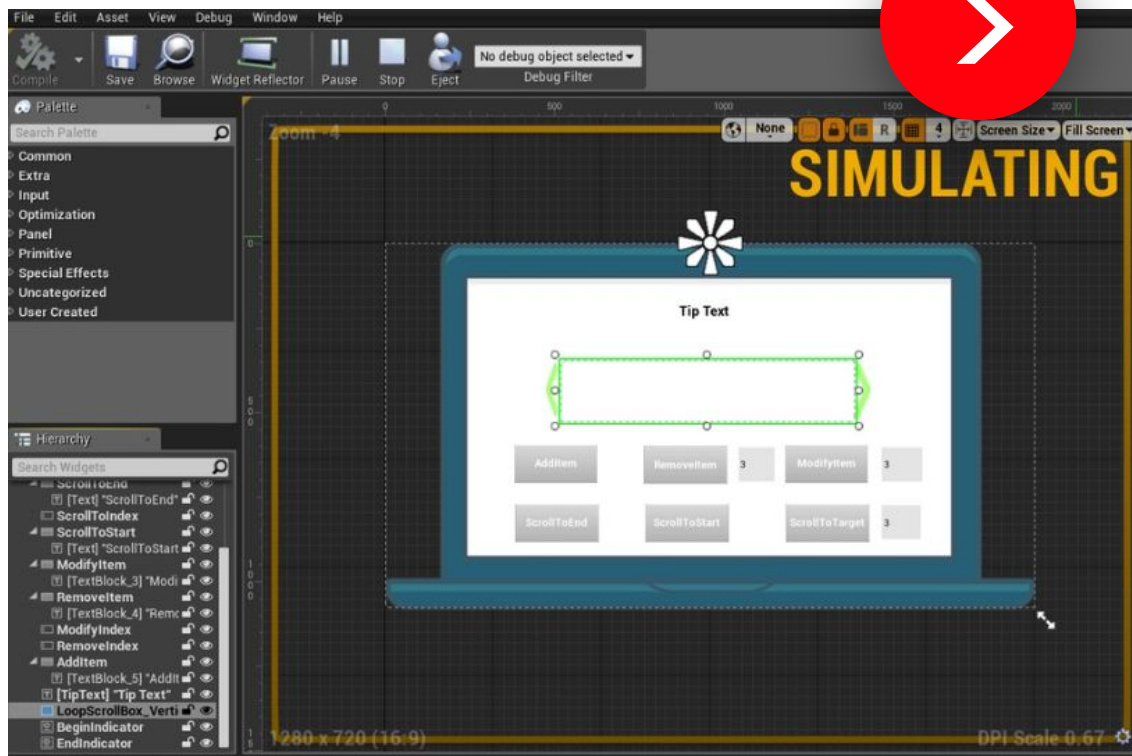
Contro

- Interamente in C++
- Animazione hardcodate
- Non si basa su UObject
- Richiede conoscenza di *SharedPtr* e *SharedRef*
- Non utilizzabile con Blueprints
- Non utilizzato con Unreal's Reflection System, il che significa che non funziona con il sistema di Garbage Collection



UNREAL UI

UMG

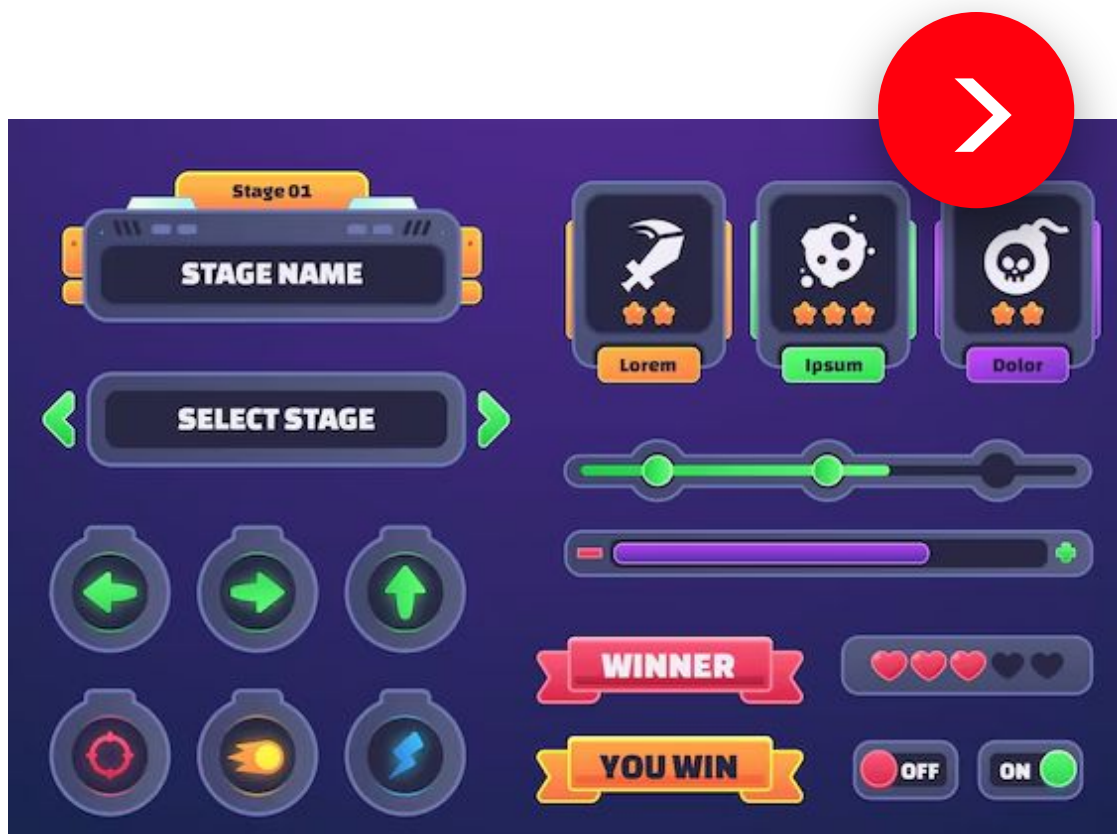


Cos'è UMG

UMG (**Unreal Motion Graphics**) deriva dalla gerarchia UObject ma non è un Actor e viene utilizzato per creare e visualizzare i widget di Slate.

In soldoni, è la versione Blueprint di Slate.

Permette sia un editing visuale all'interno di Unreal che una gestione migliore e più in linea con il resto dell'engine della parte UI.



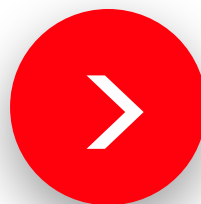
Widget

Per Widget intendiamo ogni elemento “base” di una UI.

Tutto ciò che manipoliamo in UI è considerato un widget.

Un bottone è un widget, così come lo è uno slider, una label, un blocco di testo o un blocco dove visualizziamo una texture.

Nella maggior parte dei casi tutte le nostre classi di UI ereditano come classe base da `UUserWidget` o da `UWidget`.



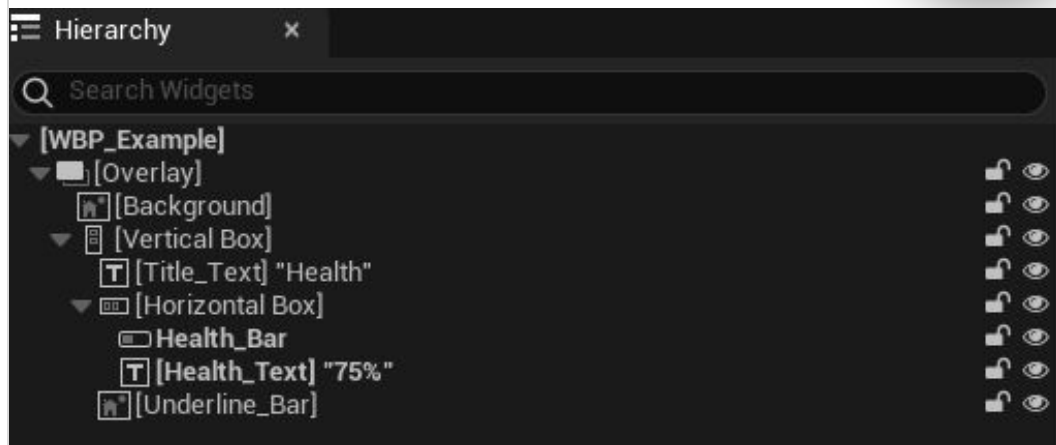
UWidget

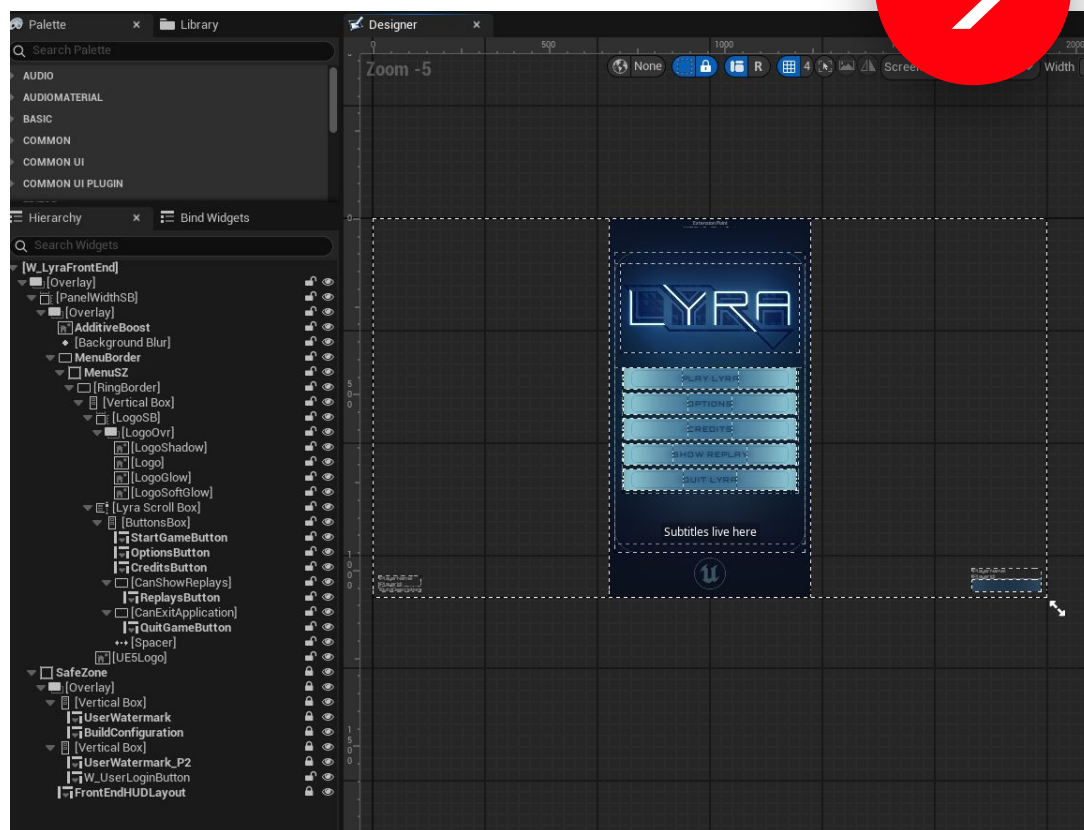
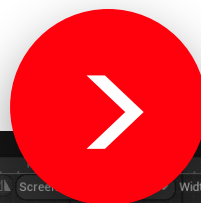
Deriva da **UVisual** che rappresenta la classe base per ogni elemento di UMG, sia Slot che Widgets

La classe base per tutti i Widget, crea gli Slate Widget e gestisce la funzionalità di routing da classi basate su Blueprint/UObject a Slate.

Questi sono widget come TextBlock, ScrollBox, Button, ecc.

Gli UWidget richiedono un Root Widget





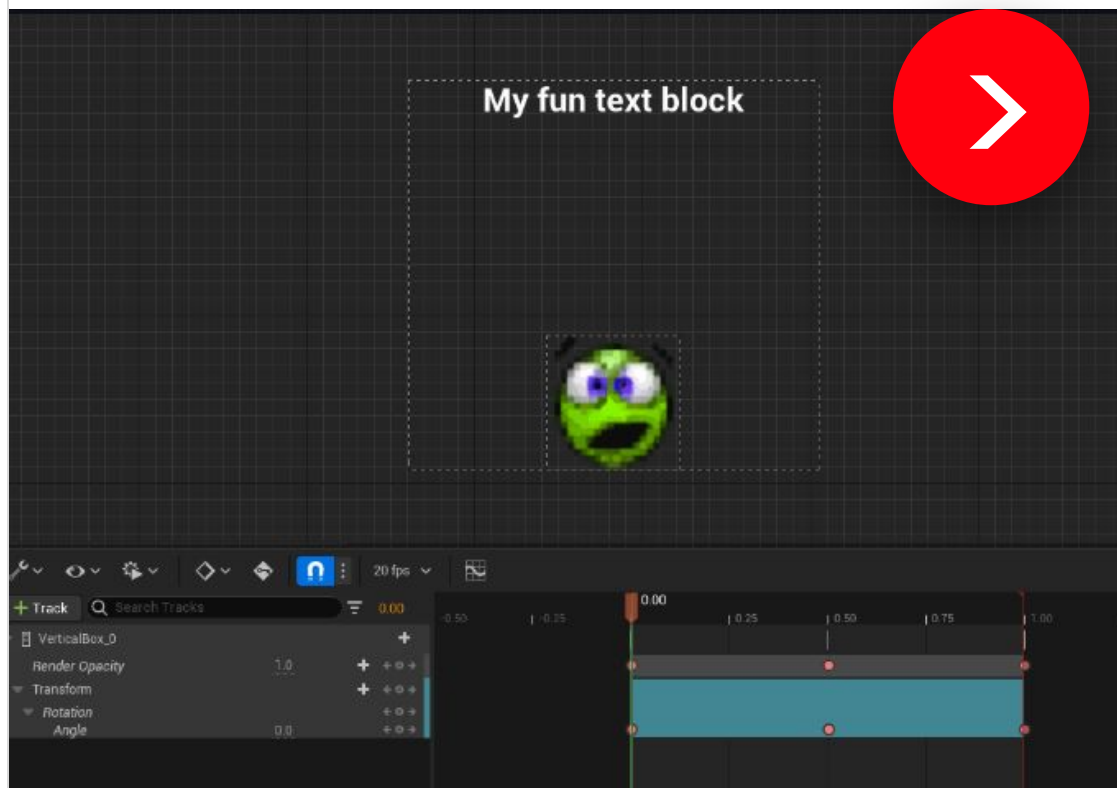
UserWidget

Gli UserWidget sono costituiti da UWidget o altri UserWidget.

Non richiedono un Root Widget.

Non possono ereditare la loro gerarchia come fanno gli Actor con i componenti, ma possono ereditare le funzionalità della classe.

Possibile creare un Widget Blueprint partendo da uno UserWidget



Animation

Ogni User Widget è in grado di creare animazioni personalizzate utilizzando i widget all'interno di se stesso tramite Sequencer.

Le animazioni possono essere create all'interno del Widget Designer.

Possono essere modificate

- Transform di rendering
- Visibilità del widget
- Parametri dei materiali
- Valori runtime



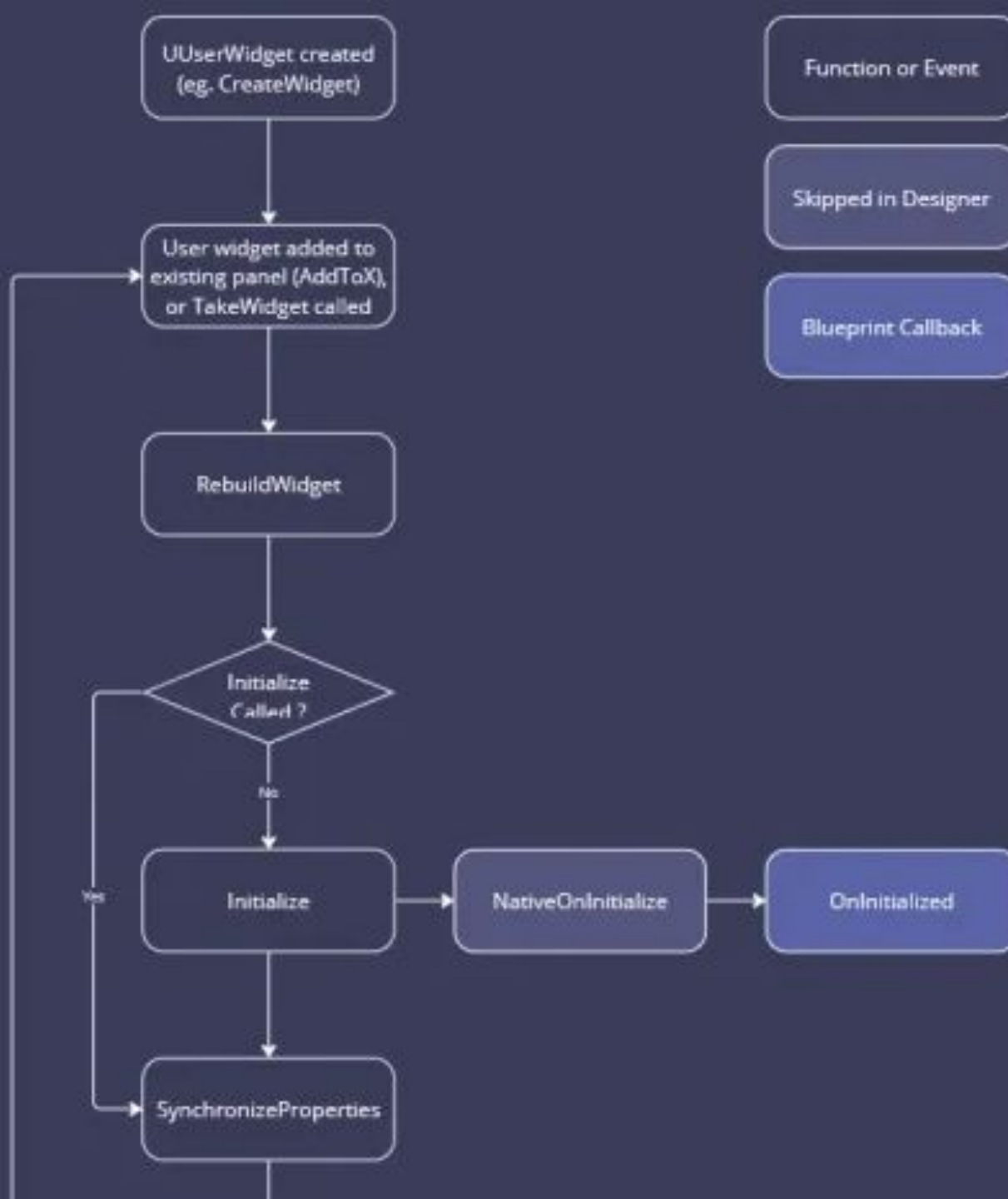
Pro e Contro

Pro

- Può essere utilizzato con Blueprints e C++
- Consente animazioni con un'interfaccia user friendly
- Ha un visual designer
- Funziona con il Garbage Collection System di Unreal
- In grado di estendere l'editor (Unreal Engine 4.23+ utilizzando Editor Utility Widgets)

Contro

- Limitato ad Unreal Engine (non utilizzabile al di fuori del motore per le normali applicazioni)
- La facilità d'uso può anche causare debiti di performance
- Può essere usato facilmente dai non-programmatori (si scherza)







Domande?



